

Rafael Christian Silva Wernesbach

**Desenvolvimento de um Sistema Educacional
Integrado para a Escola da Magistratura do
Estado do Espírito Santo**

Vila Velha/ES

2026

Rafael Christian Silva Wernesbach

Desenvolvimento de um Sistema Educacional Integrado para a Escola da Magistratura do Estado do Espírito Santo

Trabalho Preliminar de Conclusão de Curso
(TCC-1) apresentado à Unidade de Tecnologia da Universidade Vila Velha, como requisito parcial para a obtenção do grau de bacharel em Ciência da Computação.

Universidade Vila Velha

Unidade de Tecnologia

Graduação em Ciência da Computação

Orientador: Abrantes Araújo Silva Filho

Vila Velha/ES

2026

Dados Internacionais de Catalogação na Publicação (CIP)

X999x Silva Wernesbach, Rafael Christian
Desenvolvimento de um Sistema Educacional Integrado para a Escola da Magistratura do Estado do Espírito Santo / Rafael Christian Silva Wernesbach. — 2026.

45 págs. : il. color. ; 30 cm.

Orientador: Abrantes Araújo Silva Filho

Trabalho de Conclusão de Curso (Graduação em Ciência da Computação) — Universidade Vila Velha, Unidade de Tecnologia, Vila Velha/ES, 2026.

1. Tecnologia da informação aplicada à educação. 2. Gestão de processos acadêmicos. 3. Automação de processos pedagógicos. 4. Transformação digital no setor público. 5. Sistemas integrados de gestão educacional. I. Araújo Silva Filho, Abrantes. II. Universidade Vila Velha. III. Título.

Errata

Ainda não errei nada, eu acho

Rafael Christian Silva Wernesbach

Desenvolvimento de um Sistema Educacional Integrado para a Escola da Magistratura do Estado do Espírito Santo

Trabalho Preliminar de Conclusão de Curso
(TCC-1) apresentado à Unidade de Tecnologia da Universidade Vila Velha, como requisito parcial para a obtenção do grau de bacharel em Ciência da Computação.

Trabalho {a|re}provado. Vila Velha/ES, xx de xxx de 2021.

Abrantes Araújo Silva Filho
Orientador

NOME DO PROFESSOR
Membro da banca

NOME DO PROFESSOR
Membro da banca

Vila Velha/ES
2026

Resumo

Este trabalho apresenta o desenvolvimento, a modelagem e a análise do sistema Educa EMES, concebido para atender às demandas de gestão acadêmica e administrativa da Escola da Magistratura do Espírito Santo (EMES). O estudo parte da identificação de limitações associadas aos processos manuais anteriormente adotados pela instituição e à insuficiência de soluções previamente utilizadas para apoiar o fluxo pedagógico. Nesse contexto, o sistema é examinado como uma solução integrada voltada à centralização e à automação de atividades relacionadas, principalmente, à inscrição em cursos, ao controle de frequência e à emissão de certificados, com suporte complementar a processos administrativos. Metodologicamente, o trabalho caracteriza-se como pesquisa aplicada, desenvolvida por meio de estudo de caso, tomando como objeto um sistema real implementado em ambiente de produção. A análise contempla aspectos de arquitetura, modelagem e aderência aos processos institucionais, buscando evidenciar como a integração entre módulos e a digitalização das rotinas contribuem para maior eficiência operacional, melhoria no controle das informações e fortalecimento da gestão educacional. Assim, o Educa EMES é apresentado como exemplo de sistema integrado capaz de apoiar a modernização dos processos acadêmicos em instituições públicas de formação profissional.

Palavras-chave: Sistemas de informação educacional, gestão educacional, automação de processos acadêmicos, sistemas integrados, modernização de instituições de ensino..

Abstract

This study presents the development, modeling, and analysis of the Educa EMES system, designed to meet the academic and administrative management demands of the Escola da Magistratura do Espírito Santo (EMES). The study is based on the identification of limitations associated with the manual processes previously adopted by the institution and the insufficiency of earlier solutions used to support the pedagogical workflow. In this context, the system is examined as an integrated solution aimed at centralizing and automating activities mainly related to course enrollment, attendance control, and certificate issuance, with complementary support for administrative processes. Methodologically, the study is characterized as applied research developed through a case study, taking as its object a real system implemented in a production environment. The analysis encompasses architectural aspects, system modeling, and adherence to institutional processes, seeking to demonstrate how integration between modules and the digitalization of routines contribute to greater operational efficiency, improved information control, and the strengthening of educational management. Thus, Educa EMES is presented as an example of an integrated system capable of supporting the modernization of academic processes in public professional education institutions.

Keywords: Educational information systems, educational management, academic process automation, integrated systems, modernization of educational institutions..

Lista de abreviaturas e siglas

ABNT	Associação Brasileira de Normas Técnicas
STEM	Science, Technology, Engineering, Mathematics
EMES	Escola da Magistratura do Estado do Espírito Santo

Sumário

1	INTRODUÇÃO	15
1.1	Descrição do problema	15
1.2	Formulação do problema	16
1.3	Proposta de solução	16
1.4	Objetivos	17
1.4.1	Objetivo geral	17
1.4.2	Objetivos específicos	17
1.5	Justificativa	17
2	REVISÃO BIBLIOGRÁFICA	19
2.1	Arquitetura de Software e Engenharia de Sistemas	19
2.2	PHP	20
2.3	Laravel e Model-View-Controller	20
2.4	Banco de Dados Relacional e MySQL	20
2.5	Padrões de Design e Arquitetura	21
2.5.1	Factory Pattern	21
2.5.2	Strategy Pattern	21
2.5.3	Builder Pattern	21
2.5.4	Service Layer	22
2.6	Templating e Rasterização de Views	22
2.7	Componentes Reativos e Livewire	22
2.8	Painéis Administrativos: Filament	23
2.9	Automação de Processos e Orquestração	23
2.10	Processamento Assíncrono e Job Queues	24
2.11	Commands e Artisan Console	24
2.12	Arquitetura Orientada a Eventos	24
3	METODOLOGIA	27
3.1	Natureza da pesquisa	27
3.2	Abordagem metodológica	27
3.3	Contexto e objeto do estudo	27
3.4	Etapas de análise do fluxo pedagógico	28
3.4.1	Análise do fluxo pedagógico atual da EMES no contexto de utilização do sistema Educa EMES	28
3.4.2	Identificação dos pontos de intervenção manual e as limitações de automação existentes nas etapas do processo acadêmico	29

3.4.3	Identificação dos gargalos e as oportunidades de melhoria no fluxo pedagógico da EMES	29
3.5	Análise do Sistema Atual	30
3.6	Análise do Sistema Atual	30
3.6.1	Arquitetura tecnológica da aplicação	30
3.6.2	Modelo de persistência e banco de dados	31
3.6.3	Entidades centrais do fluxo pedagógico	32
3.6.4	Relacionamentos relevantes para a automação	33
3.6.5	Limitações estruturais observadas	35
3.7	Levantamento de requisitos	36
3.8	Modelagem da solução proposta	36
3.9	Implementação da solução proposta	36
	REFERÊNCIAS	37
	APÊNDICES	39
	APÊNDICE A – TÍTULO DO APÊNDICE	41
	ANEXOS	43
	ANEXO A – TÍTULO DO ANEXO	45

1 Introdução

A transformação digital no setor público tem impulsionado a revisão de processos administrativos e educacionais, especialmente em instituições cuja atuação depende de controle operacional, rastreabilidade das informações e padronização de rotinas. No contexto da formação profissional, a adoção de sistemas integrados se mostra relevante para apoiar atividades como planejamento de cursos, gestão de inscrições, acompanhamento de participação, registro de frequência e certificação. Além de outras etapas administrativas que permeiam o ciclo pedagógico.

Nesse cenário, escolas da magistratura desempenham papel estratégico na qualificação de magistrados, servidores e demais públicos externos ou vinculados ao Poder Judiciário, o que demanda instrumentos tecnológicos compatíveis com a complexidade de seus fluxos institucionais. A Escola da Magistratura do Espírito Santo (EMES) insere-se nesse contexto ao reunir atividades acadêmicas, administrativas e comunicacionais que precisam operar de forma articulada, segura e eficiente.

Com o objetivo de apoiar essas demandas, foi desenvolvido e lançado em novembro de 2025 o sistema Educa EMES, uma solução integrada de gestão educacional construída em contexto real de aplicação institucional e atualmente utilizada. O sistema passou a concentrar diferentes módulos relacionados à gestão acadêmica e administrativa, oferecendo suporte a processos como inscrição em cursos, controle de frequência, emissão de certificados, mensageria, documentação e operações internas da instituição.

Embora o Educa EMES tenha representado um avanço significativo na modernização e integração dos processos da EMES, o fluxo pedagógico ainda apresenta diversas etapas com intervenção manual, ausência de padronização em determinadas rotinas e limitações quanto à automação integral do ciclo acadêmico. Diante disso, este trabalho não se limita à descrição do sistema já em uso, mas sim na proposta, modelagem e implementação de uma expansão do sistema capaz de consolidar a automação do fluxo pedagógico no sistema existente. Nessa perspectiva, o Educa EMES é tratado simultaneamente como ambiente base da solução proposta e como estudo de caso aplicado à gestão educacional em instituição pública.

1.1 Descrição do problema

Antes da adoção de uma solução integrada, a EMES executava parte relevante de seus processos pedagógicos por meio de procedimentos manuais. Entre essas rotinas estavam registros físicos de frequência, controles operacionais dispersos e atividades dependentes

de intervenção direta dos servidores, o que reduzia a eficiência das ações pedagógicas e dificultava a consolidação das informações acadêmicas.

Em 2023, a instituição passou a utilizar o sistema Emeron Web como alternativa de apoio ao fluxo pedagógico. Contudo, a solução apresentava limitações relacionadas à usabilidade, à adequação arquitetural e ao escopo funcional, atendendo apenas parte das necessidades da escola. Como consequência, diversas atividades continuaram sendo realizadas de maneira manual, como comunicação com alunos, envio de mensagens, divulgação de cursos e aplicação de formulários.

A implementação do Educa EMES representou um avanço significativo ao integrar processos antes fragmentados e fornecer uma base tecnológica mais aderente ao contexto institucional. Ainda assim, verificou-se que a existência de um sistema em produção não eliminou completamente os pontos de intervenção manual no fluxo pedagógico. Persistem lacunas relacionadas à automação de etapas, à padronização de procedimentos e à redução da dependência de operações executadas manualmente ao longo do ciclo acadêmico.

Desse modo, o problema abordado neste trabalho não consiste na inexistência de um sistema para a EMES, mas na necessidade de ampliar a automação do fluxo pedagógico dentro de uma plataforma já implantada. Essa necessidade envolve identificar os gargalos e limitações do fluxo atual, propor melhorias baseadas em automação e implementar mecanismos que aumentem a eficiência operacional, a consistência dos procedimentos das atividades acadêmicas.

1.2 Formulação do problema

Como desenvolver uma solução capaz de ampliar e consolidar a automação do fluxo pedagógico no sistema Educa EMES, reduzindo intervenções manuais e pontos de gargalos, porém ainda permitindo a intervenção manual e a rastreabilidade de todo o fluxo, promovendo maior eficiência e padronização dos processos acadêmicos da EMES?

1.3 Proposta de solução

Como resposta ao problema identificado, este trabalho propõe a evolução do sistema Educa EMES por meio do desenvolvimento, da modelagem e da implementação de mecanismos voltados à automação do fluxo pedagógico. A proposta considera o sistema já existente como base tecnológica da solução, concentrando-se principalmente nos processos de inscrição, acompanhamento de frequência, conclusão de cursos, emissão de certificados e geração de relatórios.

A proposta adotada corresponde a uma pesquisa aplicada, uma vez que a solução é concebida a partir de uma demanda institucional concreta e avaliada em um ambiente

real de uso. Nesse contexto, a contribuição do trabalho está na identificação de lacunas e gargalos do fluxo atual, na proposição de uma modelagem aderente às necessidades da EMES, na implementação dos mecanismos de automação e na análise qualitativa dos impactos observados a partir dessa evolução.

Assim, o Educa EMES não é tratado como resultado final e acabado, mas como uma plataforma integrada em processo de melhoria contínua. A solução proposta busca consolidar o fluxo pedagógico dentro do sistema, ampliar o nível de automação das rotinas acadêmicas e reduzir a dependência de procedimentos manuais ainda existentes na operação institucional.

1.4 Objetivos

1.4.1 Objetivo geral

Desenvolver uma solução de automação do fluxo pedagógico no sistema Educa EMES.

1.4.2 Objetivos específicos

- Analisar o fluxo pedagógico atual da EMES no contexto de utilização do sistema Educa EMES.
- Identificar os pontos de intervenção manual e as limitações de automação existentes nas etapas do processo acadêmico.
- Identificar os gargalos e as oportunidades de melhoria no fluxo pedagógico da EMES.
- Analisar o atual sistema para contextualizar a proposta de maneira técnica.
- Definir os requisitos e as funcionalidades necessárias para ampliar a automação do fluxo pedagógico no sistema.
- Modelar a solução proposta para ampliação da automação do fluxo pedagógico.
- Implementar os mecanismos necessários para automatizar etapas do fluxo pedagógico no sistema.
- Avaliar qualitativamente os impactos da solução proposta sobre a eficiência, a padronização e o controle dos processos acadêmicos.

1.5 Justificativa

A proposta justifica-se pela relevância da automação de processos educacionais em instituições públicas que dependem de fluxos administrativos e acadêmicos confiáveis, integrados

e rastreáveis. Nesses contextos, a persistência de atividades manuais tende a gerar retrabalho, ampliar a possibilidade de inconsistências e dificultar o acompanhamento das ações realizadas ao longo do fluxo pedagógico.

No caso da EMES, a adoção do Educa EMES representou uma melhoria expressiva na digitalização dos processos institucionais, mas também evidenciou que a existência de um sistema integrado não encerra, por si só, as necessidades de evolução tecnológica da organização. A permanência de etapas dependentes de intervenção manual demonstra a importância de investigar como a automação pode ser ampliada de maneira estruturada, preservando aderência ao contexto institucional e promovendo ganhos de eficiência e padronização.

Sob a perspectiva acadêmica, a proposta contribui para a área de Sistemas de Informação ao abordar uma experiência real de evolução de software em produção, com ênfase na modelagem e implementação de mecanismos de automação aplicados à gestão educacional. Além disso, o recorte adotado nos processos de inscrição, frequência e certificação concentra-se em etapas centrais do fluxo pedagógico, o que reforça a pertinência do estudo de caso e sua potencial utilidade para contextos institucionais semelhantes.

2 Revisão Bibliográfica

Este capítulo tem como objetivo apresentar os fundamentos teóricos que englobam a proposta de automação do fluxo pedagógico no sistema Educa EMES. A revisão parte de conceitos fundamentais de engenharia de software, arquitetura de aplicações e padrões de design, avançando para elementos tecnológicos específicos (framework Laravel, banco de dados MySQL, componentes reativos Livewire, templates Blade) e finalizando com padrões de automação de processos e processamento assíncrono. A construção desse percurso teórico busca sustentar, em nível conceitual e técnico, as decisões arquiteturais, de design e de implementação discutidas nos capítulos seguintes.

2.1 Arquitetura de Software e Engenharia de Sistemas

A engenharia de software oferece fundamentos importantes para compreender que a qualidade de um sistema não depende apenas de sua existência ou de sua capacidade de executar funções isoladas, mas da forma como seus elementos são modelados, estruturados e organizados ao longo do desenvolvimento. Sommerville ([SOMMERVILLE, 2011](#)) destaca que sistemas de software devem ser construídos com atenção a atributos como manutenibilidade, confiabilidade, eficiência e adequação ao contexto de uso. De modo semelhante, Pressman e Maxim ([PRESSMAN; MAXIM, 2014](#)) observam que decisões de arquitetura, modelagem e projeto influenciam diretamente a capacidade de evolução da solução e sua aderência aos requisitos do domínio.

A separação clara de responsabilidades, redução de acoplamento e organização modular de componentes constituem princípios essenciais para sistemas capazes de evoluir sem comprometer a estabilidade. Em contextos onde a automação de processos é o objetivo central, uma arquitetura bem definida torna-se ainda mais relevante, pois precisa sustentar não apenas a execução de funcionalidades, mas também a extensibilidade da solução sem modificação de código existente ([SOMMERVILLE, 2011](#); [PRESSMAN; MAXIM, 2014](#)).

Para sistemas educacionais integrados, essa discussão é particularmente crítica. Um sistema deve representar com coerência elementos como cursos, inscrições, critérios de aprovação, registros de frequência, certificação e comunicação com participantes. Quando essa modelagem é insuficiente ou pouco aderente ao processo real, a automação tende a ser parcial, frágil ou inconsistente.

2.2 PHP

PHP (Hypertext Preprocessor) é uma linguagem de script amplamente utilizada para desenvolvimento web. Criada por Rasmus Lerdorf em 1994, PHP evoluiu para se tornar uma das linguagens mais populares para construção de aplicações web dinâmicas ([GROUP, 2024](#)). PHP é uma linguagem de propósito geral, interpretada, com tipagem dinâmica e suporte a múltiplos paradigmas de programação (procedural, orientada a objetos, funcional). Sua sintaxe é inspirada em C, Java e Perl, o que facilita a adoção por desenvolvedores familiarizados com essas linguagens.

2.3 Laravel e Model-View-Controller

Os frameworks web modernos implementam o padrão de arquitetura *Model-View-Controller* (MVC), que divide a aplicação em três camadas: Model (dados e lógica de negócio), View (visualização e apresentação) e Controller (orquestração de requisições e respostas). Essa separação é fundamental para a organização do código, permitindo que cada camada evolua de maneira independente, facilitando manutenção, testabilidade e reutilização de código ([SOMMERVILLE, 2011](#)).

Laravel ([OTWELL, 2024](#)) é um framework PHP que utiliza do MVC em sua arquitetura através de abstrações de alto nível. Stauffer ([STAUFFER, 2019](#)) descreve Laravel como uma plataforma que combina elegância sintática com potência funcional, fornecendo ferramentas integradas para roteamento, persistência de dados (via Eloquent ORM), validação, autenticação e autorização. A filosofia do framework prioriza a experiência do desenvolvedor, se apresentando como uma solução rapidamente escalável, modular e fácil de manter, permitindo focagem nas regras de negócio.

Um aspecto central do Laravel é o *Eloquent Object-Relational Mapping* (ORM) ([OTWELL; COMMUNITY, 2024c](#)), responsável por abstrair o acesso ao banco de dados relacional, permitindo que desenvolvedores trabalhem diretamente com objetos PHP, sem a necessidade de escrever SQL diretamente. Isso facilita modelagem, construção de relacionamentos e consultas. Além disso, Eloquent suporta eventos de ciclo de vida (creating, created, updating, deleting, etc.), permitindo logics customizadas durante o ciclo de vida dos modelos.

2.4 Banco de Dados Relacional e MySQL

O modelo relacional de dados, formalizado por Date e Codd, permanece como paradigma com maior adesão para armazenamento estruturado. Os bancos relacionais organizam as informações em tabelas que podem portar relações com linhas e colunas, permitindo consultas via Structured Query Language (SQL).

MySQL ([COMMUNITY, 2024](#)) é um sistema de gerenciamento de banco de dados (SGBD) relacional open source amplamente utilizado e difundido. Oferece suporte a transactions, constraints para integridade referencial, triggers, procedimentos, além de índices para otimização de consultas. Para sistemas de gestão educacional que dependem de confiabilidade, consistência e conformidade com normas de integridade, MySQL é uma boa escolha.

A modelagem adequada do esquema de dados é fundamental. Boas práticas incluem normalização com suas formas normais, definição clara de relacionamentos, uso apropriado de tipos de dados e criação de índices em colunas de busca frequente. Quando utilizadas de maneira oportuna e correta, podemos garantir que o banco de dados além de permitir consultas eficientes será íntegro.

2.5 Padrões de Design e Arquitetura

Padrões de design são soluções reutilizáveis para problemas recorrentes em design de software. Gamma e colaboradores ([GAMMA et al., 1994](#)), através do seminal “Gang of Four”, categorizam padrões em três grupos: criacionais (construção de objetos), estruturais (composição de objetos) e comportamentais (comunicação entre objetos).

2.5.1 Factory Pattern

O padrão Factory encapsula a criação de objetos, permitindo que o cliente não conheça as subclasses concretas. Isso é particularmente útil quando a instanciação depende de decisões em tempo de execução ou quando se deseja extensibilidade sem modificação de código cliente. Em contextos de automação, factory facilita a criação dinâmica de estratégias baseada em configuração.

2.5.2 Strategy Pattern

O Strategy define uma família de algoritmos, encapsula cada um e os torna intercambiáveis. Permite que o algoritmo varie independentemente do cliente que o utiliza. Em sistemas de automação de fluxos, cada etapa pode possuir múltiplas estratégias de execução (elegibilidade, validação, ação) que são selecionadas em tempo de execução conforme configuração ou estado do processo.

2.5.3 Builder Pattern

O Builder separa a construção de um objeto complexo de sua representação, permitindo que o mesmo processo construa representações diferentes. Útil para estruturas que possuem múltiplos componentes opcionais ou que precisam ser construídas gradualmente. Em

automação, builder é apropriado para construção de estruturas de análise, resultado e diagnóstico com componentes variáveis.

2.5.4 Service Layer

O padrão Service Layer encapsula lógica de negócio em serviços que atuam como fachada entre controllers (orquestração de requisição) e repositórios (acesso a dados). Martin Fowler ([FOWLER, 2024b](#)) identifica que esse padrão favorece testabilidade, reutilização de lógica entre múltiplos endpoints e clareza no fluxo de execução. Serviços podem ser compostos, permitindo que lógica complexa seja construída a partir de componentes menores.

2.6 Templating e Rasterização de Views

Em arquiteturas web MVC, a camada de apresentação precisa ser renderizada dinamicamente na base de dados e estado da aplicação. Blade ([OTWELL; COMMUNITY, 2024b](#)) é o mecanismo de templating do Laravel que permite adicionar lógica controlada (condicionais, loops, includes) em templates HTML sem perder legibilidade. Blade compila templates para PHP em cache, fazendo com que não haja overhead de parsing em requisições subsequentes.

O uso de templates bem estruturados favorece separação entre lógica de apresentação e lógica de negócio, reduz duplicação de HTML e facilita manutenção de interfaces quando regras de apresentação mudam. Em interfaces administrativas complexas, templates reutilizáveis tornam-se essenciais para eficiência de desenvolvimento.

2.7 Componentes Reativos e Livewire

Aplicações web modernas frequentemente exigem interatividade sem recarregar a página, historicamente alcançado através de JavaScript (cliente) e requisições AJAX (assíncrono). Livewire ([PROJECT, 2024](#)) é uma biblioteca PHP que traz reatividade “full-stack”, permitindo que componentes PHP reajam a eventos do usuário sem necessidade explícita de JavaScript.

Livewire possibilita que desenvolvedores escrevam componentes com estado (properties) que, ao serem modificados, automaticamente atualizam a view sem recarregar a página. Isso reduz a distância entre servidor e cliente, eliminando a necessidade de APIs REST intermediárias para operações simples, ao mesmo tempo que mantém a lógica no servidor onde pode ser testada e auditada.

Para interfaces de automação onde operações são disparadas por ações de usuário (iniciar um passo, confirmar elegibilidade, capturar diagnósticos), Livewire oferece abstração

apropriada reduzindo complexidade de JavaScript enquanto mantém responsividade.

2.8 Painéis Administrativos: Filament

Sistemas integrados de gestão frequentemente requerem interfaces administrativas robustas para configuração, monitoramento e operação. Filament ([HARRIN, 2024](#); [HARRIN; COMMUNITY, 2024](#)) é uma coleção de componentes full-stack para Laravel que acelera a construção de painéis administrativos profissionais.

Filament fornece abstrações de alto nível para CRUD (Create, Read, Update, Delete), tabelas interativas, filtros, ações em lote e customização visual. Reduz significativamente tempo de desenvolvimento de interfaces administrativas mantendo qualidade e consistência visual. Para sistemas educacionais que necessitam de várias interfaces específicas de diferentes usuários (administradores de sistema, gestores educacionais, coordenadores, instrutores), Filament possibilita construção rápida sem perder controle sobre modelagem e lógica de negócio ([HARRIN; COMMUNITY, 2024](#)).

2.9 Automação de Processos e Orquestração

A automação de processos organizacionais refere-se ao uso de mecanismos sistêmicos para executar, encadear ou apoiar atividades de maneira padronizada, reduzindo a dependência de intervenções manuais e aumentando a previsibilidade da operação. Essa discussão se aproxima do campo de *Business Process Management* (BPM), no qual processos são tratados como estruturas organizáveis, modeláveis, executáveis e passíveis de melhoria contínua ([WESKE, 2024](#)).

Segundo Weske ([WESKE, 2024](#)), a gestão por processos permite compreender a organização para além de funções isoladas, concentrando a análise no fluxo de atividades, nas regras de negócio e nas dependências entre eventos. Essa perspectiva é essencial quando se busca automatizar rotinas: automação não é digitalização de tarefas individuais, mas modelagem coerente do processo, definição clara de estados, validações, gatilhos e critérios de transição capazes de tornar a execução consistente e controlável.

Entre benefícios estão redução de falhas operacionais, rastreabilidade ampliada, padronização de rotinas e ganhos de eficiência. Porém, processos mal modelados ou pouco aderentes ao contexto podem produzir novos gargalos. A automação do fluxo pedagógico, portanto, não é meramente implementação de código, mas atividade que envolve engenharia de processos e tradução adequada das regras institucionais para a estrutura do sistema.

2.10 Processamento Assíncrono e Job Queues

Em sistemas onde operações podem ser computacionalmente intensivas ou precisam executar em background sem bloquear a requisição HTTP, processamento assíncrono torna-se essencial. Laravel oferece mecanismos integrados para isso: Queues ([OTWELL; COMMUNITY, 2024d](#)) e Tasks Scheduling ([OTWELL; COMMUNITY, 2024e](#)).

Queues permitem que tarefas sejam adicionadas a fila (em memória ou persistidas em redis/banco de dados) e processadas por workers em background. Isso desacopla o tempo de enfileiramento do tempo de processamento, permitindo que requisições HTTP retornem rapidamente ao usuário enquanto o trabalho pesado ocorre posteriormente. Scheduling, por sua vez, permite que comandos sejam executados periodicamente (ex: a cada minuto, hora ou dia) sem necessidade de cron jobs scripts externos.

Para automação de fluxos pedagógicos, esses mecanismos são relevantes quando passos de automação demandam processamento pesado, notificações múltiplas ou execução em horários específicos. Por exemplo, processamento de certificações em lote, envio de notificações, geração de relatórios ou sincronização com sistemas externos pode ser delegado a filas, deixando endpoints HTTP livres para requisições interativas ([OTWELL; COMMUNITY, 2024d](#)).

2.11 Commands e Artisan Console

Artisan ([OTWELL; COMMUNITY, 2024a](#)) é a interface de linha de comando do Laravel, fornecendo uma série de comandos built-in para gerenciamento de aplicação (criar controllers, modelos, migrations, etc.) e um framework para criação de comandos customizados.

Commands customizados encapsulam rotinas operacionais que precisam ser executadas fora do contexto de uma requisição HTTP. Exemplos incluem importação/exportação de dados, sincronização com sistemas externos, limpeza de cache, processamento em lote e orquestração de fluxos de automação.

Quando um Command é despachado (seja manualmente via CLI ou via Scheduler), ele executa em um contexto isolado com acesso total ao container de serviços, banco de dados e demais infraestrutura da aplicação. Para automação de fluxos, Commands atuam como ponto de entrada principal: buscar entidades elegíveis para processamento, aplicar regras de elegibilidade, disparar ações e coordenar orquestração de passos.

2.12 Arquitetura Orientada a Eventos

Em sistemas complexos onde múltiplos componentes precisam reagir ao mesmo evento sem acoplamento direto, o padrão Observer (ou Event-Driven) oferece solução limpa. Martin

Fowler ([FOWLER, 2024a](#)) descreve Event Sourcing como abordagem em que eventos significativos do domínio são capturados, persistidos e podem ser replicados para múltiplos subscribers.

Em Laravel, eventos podem ser disparados quando ocorrem transições significativas no domínio (ex: inscrição criada, aprovação concedida, certificação gerada). Listeners podem estar registrados para disparar ações adicionais (enviar email, atualizar dashboard, disparar outro processo) sem que o código que dispara o evento conheça os listeners ([OTWELL; COMMUNITY, 2024c](#)).

Para automação de fluxos pedagógicos, essa abordagem é particularmente apropriada: quando uma entidade transiciona de estado (ex: curso muda de “pendente aprovação” para “aprovado”), eventos podem ser disparados para notificar interessados, iniciar novos processos ou atualizar registros relacionados, sem criar acoplamento excessivo entre diferentes partes do sistema.

3 Metodologia

Este capítulo descreve a metodologia adotada no desenvolvimento do trabalho, explicando a natureza da pesquisa, a abordagem metodológica, o contexto do estudo, as etapas executadas e a estratégia de análise dos resultados. O trabalho insere-se na área de Sistemas de Informação e tem como foco a proposição e implementação de uma solução de automação do fluxo pedagógico no sistema Educa EMES, utilizado pela Escola da Magistratura do Espírito Santo (EMES).

3.1 Natureza da pesquisa

O trabalho caracteriza-se como uma pesquisa aplicada, voltada à solução de um problema concreto relacionado à gestão educacional e à automação de processos institucionais. A pesquisa aplicada busca resolver questões práticas em contextos reais, utilizando conhecimentos científicos para desenvolver soluções que atendam necessidades específicas. Nesse sentido, o estudo não se limita à análise teórica, mas envolve a identificação de lacunas no sistema existente, a modelagem de melhorias e a implementação de mecanismos que ampliem a automação do fluxo pedagógico, com avaliação dos impactos observados em ambiente institucional.

3.2 Abordagem metodológica

A abordagem metodológica adotada é qualitativa, pois o interesse do trabalho está na compreensão dos impactos da solução proposta sobre os processos pedagógicos da EMES, observando aspectos como integração entre etapas, padronização de rotinas, redução de intervenções manuais e eficiência operacional. A análise qualitativa permite explorar fenômenos complexos em contextos reais, sem depender de mensuração estatística formal, priorizando a interpretação de dados observacionais e a avaliação subjetiva de melhorias institucionais.

3.3 Contexto e objeto do estudo

A Escola da Magistratura do Espírito Santo (EMES) é uma instituição vinculada ao Poder Judiciário, responsável pela formação e capacitação de magistrados, servidores e demais públicos externos ou vinculados ao judiciário. O sistema Educa EMES, desenvolvido em contexto real de aplicação institucional, constitui uma solução integrada de gestão educacional que abrange módulos de gestão acadêmica, administrativa, contratação, orçamento,

documentação, mensageria e comunicação. O sistema já contempla funcionalidades básicas do fluxo pedagógico, como inscrição em cursos, controle de frequência, aprovação e emissão de certificados, mas apresenta limitações quanto à automação completa do ciclo acadêmico.

O objeto do estudo delimita-se à automação do fluxo pedagógico dentro do sistema Educa EMES, tratando-o simultaneamente como ambiente base da solução proposta e como estudo de caso aplicado à gestão educacional em instituição pública. O foco concentra-se nos processos de inscrição, frequência, aprovação, certificação e geração de relatórios, com ênfase na redução de intervenções manuais e na padronização de procedimentos.

3.4 Etapas de análise do fluxo pedagógico

As etapas do desenvolvimento do trabalho foram executadas de forma sequencial, alinhadas aos objetivos específicos definidos no capítulo anterior. A seguir, são descritas as três primeiras etapas realizadas.

3.4.1 Análise do fluxo pedagógico atual da EMES no contexto de utilização do sistema Educa EMES

Esta etapa consistiu na análise detalhada do fluxo pedagógico atual da EMES, considerando o contexto de utilização do sistema Educa EMES. O fluxo pedagógico foi mapeado a partir da observação das rotinas institucionais e da interação com o sistema existente, identificando as etapas principais do ciclo de vida de um curso ou evento educacional.

O processo inicia-se com o planejamento do curso pelo coordenador pedagógico, que insere informações básicas em uma planilha de cronograma, incluindo data, ministrante, local e título. Em seguida, o curso é cadastrado no sistema Educa EMES com dados mais completos, como horários exatos, carga horária, divisão de atividades, critérios de certificação, tolerâncias para frequência, informações sobre instrutores, local de realização, modalidade, senhas ou links de acesso.

Após o cadastro, ocorre a abertura e encerramento de inscrições, análise dos pré-inscritos e confirmação das inscrições com base em critérios como setor ou vínculo com o Poder Judiciário. A confirmação automática gera registros de frequência e dispara e-mails de confirmação aos alunos. Conforme o curso se aproxima, o sistema envia lembretes automáticos por e-mail (3 e 1 dia antes) às 6h da manhã.

Durante a execução do curso, os alunos registram frequência, e ao término, um servidor da EMES utiliza uma ferramenta do sistema para aprovar ou reprovar alunos conforme critérios de aprovação e presença. Posteriormente, outro servidor gera certificados para alunos e instrutores, considerando modalidades como certificado por atividade, para o evento completo ou com carga horária somada das atividades participadas.

Após a geração, os alunos recebem e-mail com link para formulário de avaliação, e ao preenchê-lo, o certificado é disponibilizado na área do aluno. O ciclo encerra-se com a geração de relatórios mensais para acompanhamento de metas, incluindo cursos realizados, número de inscritos, taxa de evasão e magistrados capacitados, mediante exportação manual de dados do sistema.

Esta análise permitiu compreender a estrutura atual do fluxo pedagógico, identificando pontos de integração com o sistema Educa EMES e estabelecendo base para as etapas subsequentes.

3.4.2 Identificação dos pontos de intervenção manual e as limitações de automação existentes nas etapas do processo acadêmico

Esta etapa envolveu a identificação sistemática dos pontos de intervenção manual e das limitações de automação existentes no fluxo pedagógico, com base na análise realizada na etapa anterior. Foram observadas as rotinas que ainda dependem de ação direta dos servidores, mesmo com o sistema Educa EMES em operação, focando nas limitações que não podem ser automatizadas ou seriam muito trabalhosas para pouco ganho.

O principal ponto de limitação identificado refere-se ao cadastro inicial do curso no sistema, durante a transição da planilha de cronograma para o Educa EMES. Essa tarefa apresenta complexidade para automação, pois envolve decisões contextuais e contato com terceiros para obtenção de informações adicionais, tornando a automatização completa pouco viável. No entanto, foi identificado que a validação da integridade do cadastro pode ser monitorada pelo sistema, verificando se os campos mínimos necessários estão preenchidos para evitar pendências durante a execução do curso.

3.4.3 Identificação dos gargalos e as oportunidades de melhoria no fluxo pedagógico da EMES

Esta etapa concentrou-se na identificação dos gargalos existentes no fluxo pedagógico e na exploração de oportunidades de melhoria, fundamentada nas análises das etapas anteriores. Os gargalos foram classificados em categorias principais, com ênfase em problemas recorrentes que impactam a eficiência operacional e podem ser endereçados através de automação.

O primeiro gargalo refere-se à integridade do cadastro e conflitos de informações, onde é comum o esquecimento de campos essenciais durante o cadastro, como senhas para salas remotas ou datas de inscrição. Isso resulta em cadastros inconsistentes com a carga horária ou agendamentos conflitantes, gerando confusão durante a execução do curso e necessidade de correções posteriores.

O segundo gargalo envolve a gestão do processo de inscrição, onde servidores frequentemente esquecem de abrir ou fechar inscrições, ou cadastram datas incorretas, comprometendo o controle temporal das vagas.

O terceiro gargalo diz respeito à confirmação manual de inscrições em cursos com públicos-alvo específicos, que requerem análise individual de perfis, consumindo tempo e gerando inconsistências.

O quarto gargalo relaciona-se à aprovação de alunos e inconsistência de critérios, principalmente em eventos com múltiplas atividades, onde erros de aprovação parcial ou desentendimentos entre responsáveis pelo cadastro e aprovação geram retrabalho.

O quinto gargalo concerne à geração de certificados inconsistentes com o cadastro, onde divergências entre a modalidade projetada e a executada resultam em retrabalho e insatisfação.

Finalmente, o sexto gargalo refere-se à geração de relatórios, onde demandas pontuais sobrecarregam a equipe, exigindo interrupção de outras atividades para extração manual de dados.

Esses gargalos representam oportunidades de melhoria através da automação, como validações automáticas de cadastro, alertas para gestão de inscrições, confirmações inteligentes, padronização de critérios de aprovação, geração automática de certificados e relatórios dinâmicos, contribuindo para maior eficiência e redução de erros no fluxo pedagógico.

3.5 Análise do Sistema Atual

3.6 Análise do Sistema Atual

A análise do sistema atual teve como objetivo compreender a estrutura técnica do Educa EMES, identificando os principais componentes tecnológicos, a organização da aplicação e o modelo de persistência utilizado para sustentar o fluxo pedagógico. Essa etapa foi necessária para contextualizar a proposta de automação dentro de uma solução já existente, em funcionamento e integrada a diferentes rotinas acadêmicas e administrativas da EMES.

3.6.1 Arquitetura tecnológica da aplicação

O Educa EMES é uma aplicação web desenvolvida em PHP, utilizando o framework Laravel como base arquitetural. A escolha por Laravel permite a organização da aplicação segundo o padrão Model-View-Controller (MVC), separando responsabilidades entre regras de negócio, persistência de dados, controle de requisições e camada de apresentação. Além disso, o sistema utiliza recursos do ecossistema Laravel, como Eloquent ORM para

mapeamento objeto-relacional, migrations e models para representação das entidades do domínio, além de mecanismos de autenticação, autorização, notificações, filas e comandos de console.

Na camada de interface administrativa, o sistema utiliza o Filament, framework construído sobre Laravel, Livewire e Blade, empregado para acelerar a construção de painéis administrativos, formulários, tabelas, filtros e ações operacionais. Essa escolha é relevante para o contexto da EMES porque grande parte das rotinas pedagógicas e administrativas depende de interfaces internas de cadastro, acompanhamento, validação e execução de procedimentos pelos servidores. Dessa forma, o Filament atua como camada de operação do sistema, enquanto Laravel concentra a estrutura da aplicação e as regras de negócio.

Sob o ponto de vista metodológico, essa arquitetura é relevante porque a proposta de automação não se desenvolve como artefato isolado, mas como evolução de uma aplicação já consolidada. Isso significa que qualquer solução proposta precisa respeitar a organização existente da camada de domínio, os padrões de implementação adotados e os mecanismos já utilizados pelo sistema para processamento interno.

3.6.2 Modelo de persistência e banco de dados

O sistema utiliza banco de dados relacional, com persistência estruturada em MySQL/MariaDB. O uso de um Sistema Gerenciador de Banco de Dados relacional é adequado ao domínio analisado, uma vez que o fluxo pedagógico depende de entidades fortemente relacionadas, como cursos, atividades, inscrições, alunos, instrutores, frequências, certificados, agendas, públicos-alvo, comunicações e avaliações. A integridade dessas relações é essencial para garantir consistência entre as etapas do processo acadêmico, especialmente nos pontos em que uma ação posterior depende de registros anteriores, como a geração de frequência a partir de uma inscrição confirmada ou a emissão de certificados a partir da aprovação do aluno.

A análise da estrutura de dados do Educa EMES foi realizada a partir do esquema relacional do banco de dados e dos models da aplicação, considerando o recorte diretamente relacionado ao fluxo pedagógico e administrativo. Esse recorte incluiu as entidades responsáveis por representar cursos, atividades, inscrições, participantes, instrutores, frequências, certificados, agendas, públicos-alvo, comunicações, arquivos, avaliações e tabelas auxiliares de classificação.

Embora o banco de dados do sistema possua outras tabelas voltadas a módulos administrativos, documentais, financeiros ou estruturas legadas, tais elementos não foram considerados nesta etapa quando não apresentavam relação direta com o ciclo pedagógico analisado. Essa delimitação foi necessária para evitar que a análise estrutural se trans-

formasse em uma descrição completa de todo o banco de dados, o que não contribuiria diretamente para os objetivos deste trabalho. Assim, o foco foi direcionado às entidades que participam da criação, execução, acompanhamento e encerramento de cursos e eventos educacionais.

3.6.3 Entidades centrais do fluxo pedagógico

Figura 1 – Diagrama conceitual entre as entidades principais

Figura 2 – Diagrama lógico entre as entidades principais

A estrutura analisada demonstra que a tabela `esc_cursos` ocupa posição central no modelo de dados. Essa tabela armazena os principais dados de planejamento e execução dos cursos, incluindo informações como título, período de realização, carga horária, número de vagas, datas de inscrição, modalidade, tipo de evento, local, critérios de aprovação, liberação de inscrição, liberação de frequência e indicação de certificação. Além disso, a própria tabela contém campos voltados à automação, como `auto`, `auto_config`, `auto_steps` e `auto_status`, que permitem associar o curso ao controle automatizado das etapas do fluxo pedagógico.

No modelo atual, a entidade de curso também é utilizada para representar atividades vinculadas a um curso principal. Essa diferenciação ocorre por meio do campo `curso_id`: registros sem valor nesse campo representam cursos ou eventos principais, enquanto registros com `curso_id` preenchido representam atividades associadas a outro curso. Essa estratégia é reforçada nos models da aplicação, nos quais o model `Cursos` aplica escopo para registros sem `curso_id`, enquanto o model `CursoAtividades` utiliza a mesma tabela física, porém filtrando registros que possuem `curso_id` preenchido.

A partir da entidade central de cursos, o fluxo pedagógico se ramifica para diferentes grupos de informações. O primeiro grupo corresponde aos vínculos de participação, representados principalmente pela tabela `esc_cursoalunos`. Essa tabela relaciona usuários a cursos ou atividades, armazenando dados como situação da inscrição, data de inscrição, aprovação e observações. Ela é fundamental para representar a entrada do participante no fluxo acadêmico, servindo como base para etapas posteriores, como geração de frequência, aprovação e certificação.

O segundo grupo corresponde ao controle de frequência, representado pela tabela `esc_cursofrequencias`. Essa estrutura associa curso, aluno, agenda e situação de frequência. Dessa forma, a presença do participante não é tratada como um dado isolado, mas como um registro vinculado ao curso ou atividade e, quando aplicável, ao horário correspondente na agenda.

O terceiro grupo é formado pelas entidades de certificação, tendo como principal tabela `esc_cursocertificados`. Essa estrutura vincula uma pessoa a um curso ou atividade certificável, armazenando informações como modelo de certificado, carga horária, arquivo gerado, disponibilidade e validação. No fluxo pedagógico, essa entidade representa a etapa final do ciclo acadêmico, pois depende de informações anteriores relacionadas à inscrição, frequência e aprovação.

O quarto grupo envolve as entidades de planejamento e apoio administrativo. Nesse conjunto estão tabelas como `adm_agenda`, `esc_instrutores`, `esc_cursopublico`, `esc_recursos`, `esc_cursoarquivos` e `esc_comunicaocurso`. Essas estruturas complementam a entidade de curso com informações necessárias para sua execução, como horários, responsáveis, público-alvo, recursos necessários, arquivos de divulgação e comunicações associadas.

Tabela 1 – Agrupamento das principais entidades analisadas

Grupo	Principais tabelas	Papel no fluxo
Núcleo pedagógico	<code>esc_cursos</code>	Representa cursos, eventos e atividades educacionais.
Participação	<code>esc_cursoalunos</code> , <code>users</code> , <code>user_perfil</code>	Representa alunos, inscrições, perfis e vínculos com cursos.
Execução e frequência	<code>adm_agenda</code> , <code>esc_cursofrequencias</code>	Controla horários, agendas e presença dos participantes.
Certificação	<code>esc_cursocertificados</code> , <code>esc_certificadomodelos</code>	Registra certificados emitidos, modelos utilizados e disponibilidade.
Apoio pedagógico-administrativo	<code>esc_instrutores</code> , <code>esc_cursopublico</code> , <code>esc_recursos</code> , <code>esc_cursoarquivos</code>	Complementa o planejamento e a execução dos cursos.
Classificação e parametrização	<code>cad_tabelaapoio</code> , <code>settings</code>	Armazena classificações, códigos e parâmetros utilizados pelo sistema.
Comunicação e avaliação	<code>esc_comunicacao</code> , <code>esc_comunicaocurso</code> , <code>esc_formavaliacao</code> , <code>esc_formsubmissions</code>	Apoia comunicações institucionais e avaliação dos cursos.

3.6.4 Relacionamentos relevantes para a automação

A análise também identificou a presença de uma tabela de apoio genérica, denominada `cad_tabelaapoio`, utilizada para armazenar diferentes classificações do sistema. Essa tabela é referenciada por diversas entidades do fluxo pedagógico, servindo para representar valores como modalidade, área temática, tipo de evento, tipo de atividade, tipo de local, tipo de público, situação de inscrição e situação de frequência. Essa abordagem reduz a

necessidade de múltiplas tabelas específicas para cada tipo de classificação e concentra a parametrização das regras em um conjunto menor de estruturas.

Além das relações diretas por chaves estrangeiras, o sistema também utiliza relacionamentos polimórficos em determinados pontos da modelagem. Um exemplo relevante é a agenda, que utiliza os campos `agendaable_type` e `agendaable_id` para permitir o vínculo de horários a diferentes entidades, como cursos, demandas ou atividades. Essa solução amplia a flexibilidade do modelo, pois uma mesma estrutura de agenda pode ser reutilizada por mais de um tipo de entidade.

Figura 3 – Representação dos campos polimórficos

ADM_AGENDA			
bigint	id	PK	
bigint	atualizado_por	FK	
bigint	setor_id	FK	
varchar	titulo		
datetime	inicio		
datetime	termino		
boolean	dia_todo		
bigint	agendaable_id		ID do registro relacionado, ex: 7432
varchar	agendaable_type		Classe do model eloquent relacionado, ex: App/Models/Escola/Cursos
bigint	local_id	FK	

Do ponto de vista do fluxo pedagógico, a estrutura de dados pode ser compreendida como uma cadeia de dependências entre entidades. O curso ou atividade é inicialmente cadastrado e parametrizado; em seguida, são definidos seus públicos, instrutores, recursos, arquivos e agenda; posteriormente, os participantes são vinculados por meio das inscrições; durante a execução, são registrados os dados de frequência; ao final, ocorre a aprovação dos participantes e a geração dos certificados. Essa sequência evidencia que a automação proposta não atua sobre uma entidade isolada, mas sobre um conjunto de registros interdependentes.

Figura 4 – Fluxo conceitual dos dados no ciclo pedagógico - colocar um diagrama de estados ou bpmn talvez

□
Legenda

Tabela 2 – Relação entre etapas do fluxo pedagógico e estruturas de dados

Etapa do fluxo	Entidades envolvidas	Dados necessários
Planejamento do curso	<code>esc_cursos</code> , <code>adm_agenda</code> , <code>esc_instrutores</code>	Datas, horários, carga horária, local, instrutores e critérios.
Configuração do público	<code>esc_cursopublico</code> , <code>user_perfil</code> , <code>cad_tabelaapoio</code>	Tipos de público, perfil dos usuários e classificações.
Inscrição	<code>esc_cursoalunos</code> , <code>users</code>	Participante, curso, situação da inscrição e data de inscrição.
Controle de frequência	<code>esc_cursofrequencias</code> , <code>adm_agenda</code>	Participante, agenda, situação de frequência e observações.
Aprovação	<code>esc_cursoalunos</code> , <code>esc_cursofrequencias</code>	Frequência, critérios de aprovação e status do participante.
Certificação	<code>esc_cursocertificados</code> , <code>esc_certificadomodelos</code>	Pessoa certificada, carga horária, modelo, arquivo e disponibilidade.
Avaliação e encerramento	<code>esc_formavaliacao</code> , <code>esc_formsubmissions</code>	Formulário, respostas, participante e curso avaliado.

A estrutura observada evidencia que o modelo de dados já possui elementos suficientes para sustentar a automação gradual do fluxo pedagógico. A existência de registros específicos para inscrições, frequências, certificados, agendas, públicos e comunicações permite que as etapas do processo sejam verificadas e atualizadas de forma sistemática. Além disso, os campos de automação presentes na tabela de cursos permitem registrar configurações, estados e diagnósticos do processamento automatizado, evitando que a solução precise ser construída como um mecanismo externo desconectado da base atual.

3.6.5 Limitações estruturais observadas

Apesar de a estrutura analisada fornecer base consistente para a automação proposta, algumas limitações estruturais foram identificadas. A primeira delas está na reutilização da tabela `esc_cursos` para representar simultaneamente cursos principais e atividades vinculadas. Embora essa estratégia favoreça reaproveitamento de estrutura, ela também introduz ambiguidade semântica, pois elementos com papéis distintos no domínio passam a compartilhar a mesma representação física.

Outra limitação relevante está na centralização de múltiplas classificações na tabela genérica `cad_tabelaapoio`. Essa abordagem oferece flexibilidade e reduz a proliferação de tabelas de domínio, mas torna mais dependente da aplicação a interpretação correta de grupos, códigos e significados utilizados em filtros, validações e regras operacionais.

Também se observou que os relacionamentos polimórficos, especialmente no caso da agenda, ampliam a flexibilidade do modelo, porém reduzem a capacidade de o próprio banco garantir integridade referencial de forma explícita. Isso desloca parte importante da confiabilidade da relação para a camada de aplicação, exigindo maior rigor na implementação e na manutenção das regras de negócio.

Por fim, a própria natureza encadeada do fluxo pedagógico faz com que inconsistências em etapas iniciais, como cadastro incompleto, agenda incompatível ou parametrização inadequada do público e dos critérios de aprovação, repercutam nas etapas posteriores de frequência, certificação, comunicação e avaliação. Em razão disso, conclui-se que a automação do fluxo pedagógico depende não apenas da existência das entidades e relacionamentos, mas também da qualidade estrutural e da coerência semântica com que esses elementos são configurados no sistema.

3.7 Levantamento de requisitos

3.8 Modelagem da solução proposta

3.9 Implementação da solução proposta

Referências

- COMMUNITY, M. *MySQL Database Documentation*. 2024. Disponível em: <https://dev.mysql.com/doc>. Citado na página 21.
- FOWLER, M. *Event Sourcing*. 2024. Disponível em: <https://martinfowler.com/eaDev/EventSourcing.html>. Citado na página 25.
- FOWLER, M. *Patterns of Enterprise Application Architecture — Service Layer*. 2024. Disponível em: <https://martinfowler.com/eaCatalog/serviceLayer.html>. Citado na página 22.
- GAMMA, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. [S.l.]: Addison-Wesley, 1994. ISBN 0201633612. Citado na página 21.
- GROUP, P. *PHP: Hypertext Preprocessor*. 2024. Disponível em: <https://www.php.net>. Citado na página 20.
- HARRIN, D. *Filament — A Collection of Full-Stack Components for Laravel*. 2024. Disponível em: <https://filamentphp.com>. Citado na página 23.
- HARRIN, D.; COMMUNITY, F. *Filament Admin Panel Documentation*. 2024. Disponível em: <https://filamentphp.com/docs/3.x>. Citado na página 23.
- OTWELL, T. *Laravel — The PHP Framework for Web Artisans*. 2024. Disponível em: <https://laravel.com>. Citado na página 20.
- OTWELL, T.; COMMUNITY, L. *Laravel Documentation — Artisan Console*. 2024. Disponível em: <https://laravel.com/docs/artisan>. Citado na página 24.
- OTWELL, T.; COMMUNITY, L. *Laravel Documentation — Blade Templating Engine*. 2024. Disponível em: <https://laravel.com/docs/blade>. Citado na página 22.
- OTWELL, T.; COMMUNITY, L. *Laravel Documentation — Eloquent ORM*. 2024. Disponível em: <https://laravel.com/docs/eloquent>. Citado 2 vezes, nas páginas 20 e 25.
- OTWELL, T.; COMMUNITY, L. *Laravel Documentation — Queues*. 2024. Disponível em: <https://laravel.com/docs/queues>. Citado na página 24.
- OTWELL, T.; COMMUNITY, L. *Laravel Documentation — Task Scheduling*. 2024. Disponível em: <https://laravel.com/docs/scheduling>. Citado na página 24.
- PRESSMAN, R. S.; MAXIM, B. R. *Software Engineering: A Practitioner's Approach*. 8. ed. [S.l.]: McGraw-Hill, 2014. Citado na página 19.
- PROJECT, L. *Livewire — Full-Stack Reactivity for Laravel*. 2024. Disponível em: <https://livewire.laravel.com>. Citado na página 22.
- SOMMERVILLE, I. *Software Engineering*. 9. ed. [S.l.]: Pearson, 2011. Citado 2 vezes, nas páginas 19 e 20.

STAUFFER, M. *Laravel Up and Running: A Framework for Building Modern PHP Apps*. 2. ed. [S.l.]: O'Reilly Media, 2019. ISBN 9781492041207. Citado na página 20.

WESKE, M. *Business Process Management: Concepts, Languages, Architectures*. 4. ed. [S.l.]: Springer, 2024. ISBN 9783662695173. Citado na página 23.

Apêndices

APÊNDICE A – Título do apêndice

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent sollicitudin, ligula nec dignissim tempus, velit risus malesuada eros, eu commodo metus quam eu magna. Aenean in urna elementum, finibus tellus eget, rhoncus est. Cras at massa et velit fermentum lacinia. Suspendisse dignissim aliquet pretium. Maecenas volutpat pretium blandit. Sed vulputate efficitur libero, a elementum nisi vestibulum ut. Phasellus a semper metus. Suspendisse potenti. Pellentesque ullamcorper dui felis, vel egestas turpis tempor nec. Curabitur in lacus faucibus, lobortis risus eget, scelerisque turpis. Cras porta sollicitudin convallis.

Anexos

ANEXO A – Título do anexo

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent sollicitudin, ligula nec dignissim tempus, velit risus malesuada eros, eu commodo metus quam eu magna. Aenean in urna elementum, finibus tellus eget, rhoncus est. Cras at massa et velit fermentum lacinia. Suspendisse dignissim aliquet pretium. Maecenas volutpat pretium blandit. Sed vulputate efficitur libero, a elementum nisi vestibulum ut. Phasellus a semper metus. Suspendisse potenti. Pellentesque ullamcorper dui felis, vel egestas turpis tempor nec. Curabitur in lacus faucibus, lobortis risus eget, scelerisque turpis. Cras porta sollicitudin convallis.